

Lattice Boltzmann method: 2D flow past a cylinder

1 Serial Profiling

I profiled the serial solver on the default validation case 800×400 , $Re = 100$, $u_{in} = 0.05$, and $T = 60000$, with output disabled. The `gprof` flat profile shows that `collide()`, `stream()`, and `bounce_back()` account for 71.88%, 25.82%, and 1.98% of the sampled time. Since the MPI version parallelizes all three routines over owned rows, I use $f = 0.9968$ as the profiled parallel fraction for the Amdahl and Gustafson reference models.

The optimized output-disabled serial baseline achieved 21.20 MLUPS on the 800×400 , $T = 60000$ case, measured as the mean of five runs.

2 MPI Parallelization

2.1 Design

The MPI version uses a one-dimensional row-block decomposition. Each rank owns a contiguous set of global rows plus two ghost rows, one above and one below. Since D2Q9 streaming only needs neighbouring cells, communication is limited to the upper and lower neighbouring ranks. The distribution functions keep the original structure-of-arrays layout, so the serial collision, bounce-back, streaming, inlet, and outlet routines require only local modifications.

2.2 Communication and Overlap Strategy

Each time step first performs BGK collision and bounce-back on the owned rows. The solver then starts a non-blocking halo exchange using `MPI_Irecv` and `MPI_Isend`. While communication is in flight, each rank streams its interior rows; after `MPI_Waitall`, it streams the boundary rows using the received ghost data. For pull streaming with row decomposition, only the three populations crossing each horizontal boundary are exchanged, reducing each halo message from $9n_x$ to $3n_x$ doubles.

2.3 Strong Scaling Results and Amdahl Comparison

Strong scaling was evaluated for fixed global grids of 800×400 , 1200×600 , and 1600×800 , all with $T = 60000$ and output disabled. Speedup is measured relative to the corresponding $p = 1$ MPI run. For 1600×800 , throughput rises from 23.03 MLUPS at $p = 1$ to 3998.36 MLUPS at $p = 256$, giving $173.64\times$ speedup and 67.8% parallel efficiency. Full MPI tables are in the source archive; Figures 1 and 2 plot the derived speedup and efficiency. Using the profiled parallel fraction $f = 0.9968$, I

compare the measurements with the simple Amdahl reference

$$S_{\text{Amdahl}}(p) = \frac{1}{(1-f) + f/p}.$$

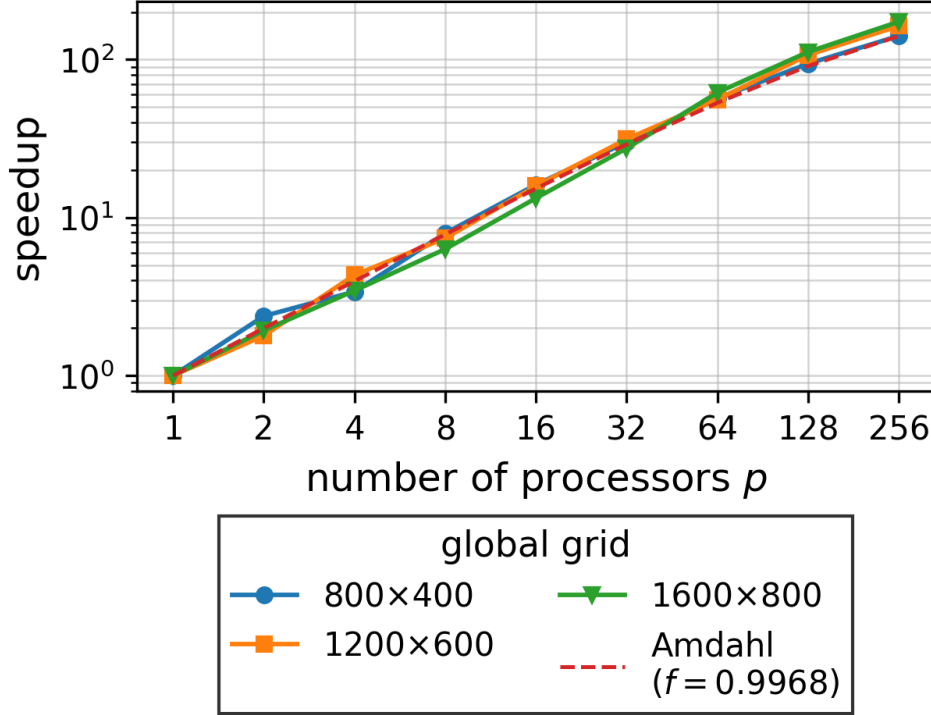


Figure 1: MPI strong scaling for three fixed global grids, with the Amdahl reference using $f = 0.9968$.

Figure 1 shows near-ideal scaling at small and moderate process counts. Some points exceed the Amdahl prediction ($\approx 141\times$ at $p = 256$) while remaining below p , so the speedup is not strictly superlinear; the excess over the model comes mainly from cache effects, the shrinking per-rank working set, and measurement noise. At high p , the smaller grid becomes communication limited, while larger grids scale better because each rank keeps more local work. At $p = 256$ the one-dimensional decomposition leaves only $400/256 \approx 1.6$ owned rows per rank on 800×400 , so the two ghost rows hold more data than the interior; this degenerate surface-to-volume ratio is the concrete reason communication reaches 70.8% for 800×400 but only 40.6% for 1600×800 .

2.4 Weak Scaling Results and Gustafson Comparison

Figure 2 shows weak scaling with a fixed local height of 400 rows per rank, so the global height grows with p . Total throughput rises from 23.87 MLUPS at $p = 1$ to 3037.79 MLUPS at $p = 256$, but efficiency drops to 49.7% as communication reaches 47.4% of runtime.

Using $\alpha = 1 - f = 0.0032$, the simple Gustafson model predicts $1 - \alpha(1 - 1/p) \approx 99.7\%$ efficiency at $p = 256$, far above the measured 49.7%. This gap shows that the model’s constant-serial-fraction assumption does not hold here: with row decomposition, each rank still exchanges halo data

proportional to n_x , and synchronization costs grow with p . Thus, halo exchange increasingly dominates at large process counts.

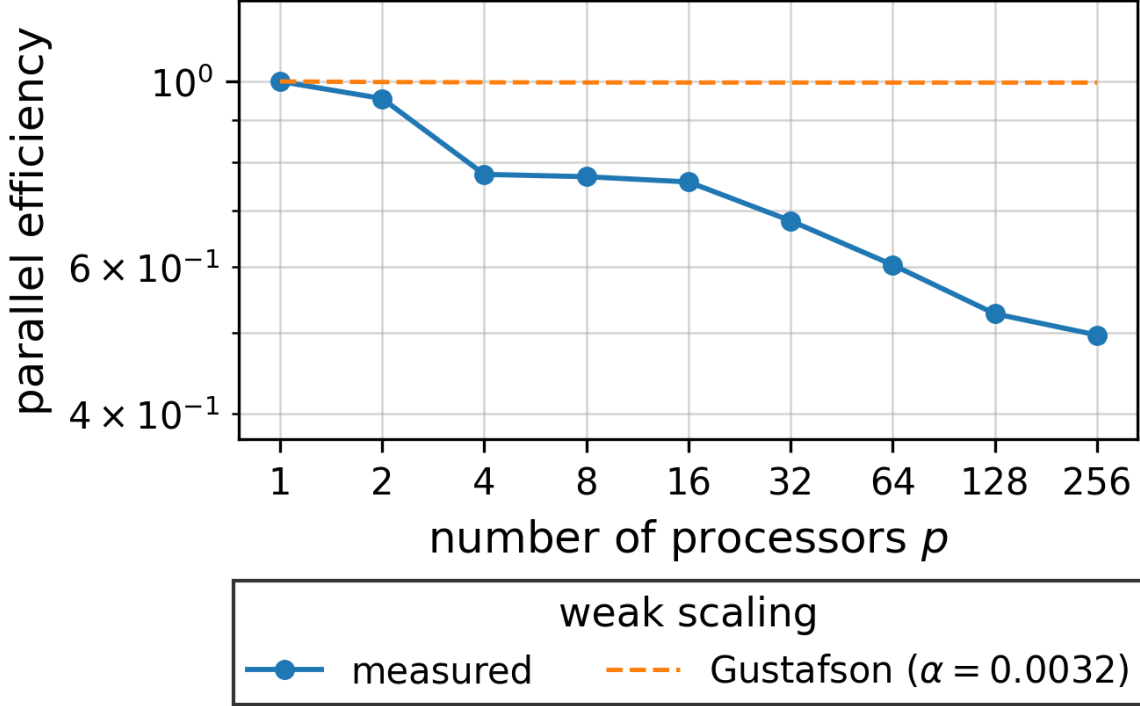


Figure 2: MPI weak-scaling efficiency with fixed local height of 400 rows per rank.

3 CUDA Acceleration

3.1 Design

The CUDA version moves the main lattice update to the GPU using a fused pull-streaming and BGK collision kernel. Each thread owns one destination cell, pulls the nine incoming D2Q9 populations from neighbouring cells into registers, computes ρ and $\mathbf{u}$, applies BGK collision, and writes the result to the next buffer. Thus, the intermediate post-collision state is not materialized in global memory.

This reduces global memory traffic, gives each thread clear ownership of its output cell, and avoids a separate streaming kernel. Two device buffers are swapped each step (double buffering). Solid boundaries use on-link bounce-back, and a small boundary kernel applies the inlet, outlet, and optional probe recording.

3.2 Memory Layout

The CUDA implementation uses a structure-of-arrays layout,

$$f[q(\text{pitch} \cdot n_y) + y \cdot \text{pitch} + x],$$

so neighbouring threads updating adjacent x -locations access contiguous memory for the same D2Q9 population. This improves coalescing compared with an array-of-structures layout. The row pitch can be padded as $pitch = \text{round_up}(n_x, \text{pitch_align})$, which improves alignment when n_x is not naturally aligned. Layout is fixed to SoA, so pitch alignment is the layout parameter I study quantitatively (Figure 4) rather than implementing a separate array-of-structures variant.

3.3 CUDA Cost Model and Roofline Estimate

The fused CUDA update is mainly memory-bandwidth limited. Each lattice update reads nine double-precision populations and writes nine updated populations, giving a lower-bound traffic estimate of $18 \times 8 = 144$ bytes per lattice update. With a nominal V100 HBM2 bandwidth of 900 GB/s, this gives a rough roofline estimate of about 6250 MLUPS. The best measured result, 4906.88 MLUPS, is below this bound but plausible because the kernel also reads the solid mask, performs boundary checks, and executes BGK arithmetic.

3.4 CUDA Performance Results

The CUDA study varies problem size, block shape, and pitch alignment. All CUDA timings use `every=0` and `probe_every=0`, so snapshot output and probe recording are excluded.

On 800×400 , the best CUDA result (3979.15 MLUPS) is $187.7\times$ the 21.20 MLUPS serial baseline. Across all tested CUDA problem sizes, the global best result is 4906.88 MLUPS on the 1500×750 grid. Full tables for all grid, block, and pitch configurations are in the source archive; Figures 3 and 4 summarize them.

Leaderboard (all CUDA runs, sorted by MLUPS)

Rank	Config	Grid	Block	Threads/block	Pitch align	Steps	Wall (s)	MLUPS
1	cuda_nx1500_ny750_bx64_by4_pa64_s60000	1500x750	64x4	256	64	60000	13.76	4906.88
2	cuda_nx1600_ny800_bx64_by4_pa32_s60000	1600x800	64x4	256	32	60000	15.67	4899.73
3	cuda_nx1600_ny800_bx64_by4_pa16_s60000	1600x800	64x4	256	16	60000	15.68	4899.09
4	cuda_nx1600_ny800_bx64_by4_pa1_s60000	1600x800	64x4	256	1	60000	15.68	4897.77
5	cuda_nx1600_ny800_bx64_by4_pa64_s60000	1600x800	64x4	256	64	60000	15.68	4897.10
6	cuda_nx1500_ny750_bx64_by4_pa16_s60000	1500x750	64x4	256	16	60000	13.94	4843.51
7	cuda_nx1500_ny750_bx64_by4_pa32_s60000	1500x750	64x4	256	32	60000	13.94	4841.47
8	cuda_nx1600_ny800_bx32_by4_pa16_s60000	1600x800	32x4	128	16	60000	15.95	4816.08
9	cuda_nx1600_ny800_bx32_by4_pa64_s60000	1600x800	32x4	128	64	60000	15.95	4815.83
10	cuda_nx1600_ny800_bx32_by4_pa32_s60000	1600x800	32x4	128	32	60000	15.95	4815.10
11	cuda_nx1600_ny800_bx32_by4_pa1_s60000	1600x800	32x4	128	1	60000	15.95	4814.79
12	cuda_nx1600_ny800_bx32_by8_pa16_s60000	1600x800	32x8	256	16	60000	15.98	4805.43
13	cuda_nx1600_ny800_bx32_by8_pa32_s60000	1600x800	32x8	256	32	60000	15.98	4805.11
14	cuda_nx1600_ny800_bx32_by8_pa64_s60000	1600x800	32x8	256	64	60000	15.98	4805.04
15	cuda_nx1600_ny800_bx32_by8_pa1_s60000	1600x800	32x8	256	1	60000	15.99	4804.39
16	cuda_nx1600_ny800_bx32_by16_pa16_s60000	1600x800	32x16	512	16	60000	16.01	4797.15
17	cuda_nx1600_ny800_bx32_by16_pa64_s60000	1600x800	32x16	512	64	60000	16.01	4796.71
18	cuda_nx1600_ny800_bx32_by16_pa1_s60000	1600x800	32x16	512	1	60000	16.01	4796.62
19	cuda_nx1600_ny800_bx32_by16_pa32_s60000	1600x800	32x16	512	32	60000	16.02	4795.09
20	cuda_nx1500_ny750_bx64_by4_pa1_s60000	1500x750	64x4	256	1	60000	14.15	4771.73
21	cuda_nx1600_ny800_bx16_by16_pa16_s60000	1600x800	16x16	256	16	60000	16.12	4765.63
22	cuda_nx1600_ny800_bx16_by16_pa32_s60000	1600x800	16x16	256	32	60000	16.12	4763.72
23	cuda_nx1600_ny800_bx16_by16_pa64_s60000	1600x800	16x16	256	64	60000	16.12	4763.66
24	cuda_nx1600_ny800_bx16_by16_pa1_s60000	1600x800	16x16	256	1	60000	16.12	4763.47
25	cuda_nx800_ny400_bx64_by4_pa32_s60000	800x400	64x4	256	32	60000	4.83	3979.15

Figure 3: Top CUDA runs sorted by MLUPS, sweeping block shape, pitch alignment, and grid size.

Block / pitch sweep for grid 1500x750

Rank	Config	Grid	Block	Threads/block	Pitch align	Steps	Wall (s)	MLUPS
1	cuda_nx1500_ny750_bx64_by4_pa64_s60000	1500x750	64x4	256	64	60000	13.76	4906.88
2	cuda_nx1500_ny750_bx64_by4_pa16_s60000	1500x750	64x4	256	16	60000	13.94	4843.51
3	cuda_nx1500_ny750_bx64_by4_pa32_s60000	1500x750	64x4	256	32	60000	13.94	4841.47
4	cuda_nx1500_ny750_bx64_by4_pa1_s60000	1500x750	64x4	256	1	60000	14.15	4771.73

Figure 4: Pitch-alignment sweep for the best CUDA block shape on the 1500×750 grid.

The leaderboard (Figure 3) shows that block shape dominates pitch: on 1600×800 , throughput falls monotonically from 4899.73 MLUPS at 64×4 to 4765.63 MLUPS at 16×16 , because wider x -direction blocks give longer contiguous, better-coalesced SoA reads. Figure 4 shows that for this best 64×4 shape, pitch padding has only a modest effect: on 1500×750 it gives about 2.8% (4906.88 vs 4771.73 MLUPS, `pitch_align` 64 vs 1). The same sweep on 1600×800 is essentially flat ($< 0.1\%$), because 1600 is already a multiple of 64, so padding is a no-op; alignment helps only when n_x is not naturally aligned.

4 Correctness Validation

Both parallel versions were validated on the 800×400 , $\text{Re} = 100$, $T = 60000$ case using the transverse velocity probe and the provided `strouhal.py` script. All MPI process counts from $p = 1$ to $p = 256$ recovered $St = 0.1600$. The CUDA version was also tested with three block shapes, 16×16 , 32×8 , and 64×4 , and three pitch alignments, 1, 32, and 64; all configurations recovered $St = 0.1600$.

Although the fused CUDA update is not bitwise-identical to the serial `collide-bounce-back-stream` sequence, it preserves the physical observable and reproduces the expected vortex street.

5 Going Further: Single-Node Hybrid MPI-CUDA Solver

As an extension beyond the required versions, I implemented a single-node hybrid MPI-CUDA solver that combines the MPI row decomposition with the aggregate memory bandwidth of two GPUs on one node. It addresses a limitation of each separate version: MPI-only computes the update on CPUs, so few ranks are slow while many ranks pay heavy halo and synchronization cost; CUDA-only confines the whole domain to one device, capping problem size and throughput at that GPU’s memory and bandwidth. The hybrid solver instead uses only a few MPI ranks, each bound to one GPU, splitting the domain across devices.

Each rank owns a row-block subdomain with two ghost rows. The local update reuses the fused CUDA pull-streaming/BGK kernel. Only the halo exchange changes: each rank exchanges the three D2Q9 populations crossing each horizontal boundary. I tested CUDA-aware MPI using device buffers directly and a host-staged mode using pinned buffers, without assuming NVLink, GPUDirect RDMA, or zero-copy transfer.

GPUs	Halo	Grid	Steps	N	Wall median (s)	MLUPS median	Speedup vs shared median 1 GPU same grid	Efficiency	Max halo ove
1	cuda	1500x750	60000	1	14.35	4703.81	1.00	100.00%	
2	cuda	1500x750	60000	1	11.42	5910.97	1.26	62.83%	
1	staged	1500x750	60000	1	14.35	4703.75	1.00	100.00%	
2	staged	1500x750	60000	1	11.53	5853.83	1.24	62.22%	
1	cuda	1600x800	60000	1	16.32	4705.69	1.00	100.01%	
2	cuda	1600x800	60000	1	12.31	6239.53	1.33	66.30%	
1	staged	1600x800	60000	1	16.32	4705.06	1.00	99.99%	
2	staged	1600x800	60000	1	12.52	6133.26	1.30	65.17%	
1	cuda	3200x1600	60000	1	59.05	5202.49	1.00	100.00%	
2	cuda	3200x1600	60000	1	33.90	9061.55	1.74	87.09%	
1	staged	3200x1600	60000	1	59.05	5202.22	1.00	100.00%	
2	staged	3200x1600	60000	1	34.06	9018.58	1.73	86.68%	
1	cuda	6400x3200	60000	1	233.73	5257.41	1.00	100.00%	
2	cuda	6400x3200	60000	1	121.21	10137.80	1.93	96.41%	
1	staged	6400x3200	60000	1	233.73	5257.35	1.00	100.00%	
2	staged	6400x3200	60000	1	122.36	10042.30	1.91	95.51%	
1	cuda	800x400	60000	1	5.33	3599.61	1.00	100.04%	
2	cuda	800x400	60000	1	6.78	2831.47	0.79	39.35%	
1	staged	800x400	60000	1	5.34	3596.87	1.00	99.96%	
2	staged	800x400	60000	1	6.52	2943.09	0.82	40.90%	

Figure 5: Single-node hybrid MPI–CUDA scaling results.

For small grids, the hybrid solver regresses: at 800×400 , the two-GPU CUDA-aware run achieves only $0.79\times$ speedup and 39.35% efficiency because halo exchange dominates the small local workload. At 1600×800 , the hybrid CUDA-aware run reaches 6239.53 MLUPS, higher than both the best single-GPU CUDA result, 4899.73 MLUPS, and the best MPI-only result, 3998.36 MLUPS, for the same grid; however, its two-GPU speedup is still only $1.33\times$. The benefit appears for larger grids: at 6400×3200 , CUDA-aware reaches 10137.80 MLUPS, $1.93\times$ speedup, and 96.41% efficiency. Host-staged exchange follows the same trend, so the main limitation is halo overhead rather than only staging cost.

AI Usage Statement

I used AI tools, mainly ChatGPT, to assist with generating initial drafts of the code, report, and presentation materials. AI was also used to help polish explanations.

All generated content was manually reviewed, modified, and verified by me. The final implementation, experiments, validation, performance analysis, and report interpretation were checked and finalized by me.